Grazioso Salvare Animal Shelter

CS 499 MILESTONE ONE CODE REVIEW



Presentation by Jennifer Joseph

PROJECT OVERVIEW

ProjectTwoDashboard.ipynb

C: > Users > jenni > AppData > Local > Temp > 56b8650f-06b2-485e-ad18-d2251cc56d5e_ProjectTwo.zip.d5e > ProjectTwoDashboard.i Jupyter Notebook

Generate + Code + Markdown | Run All ··· Select Kernel

```python
# Setup the Jupyter version of Dash
from jupyter_dash import JupyterDash

# Configure the necessary Python module imports for dashboard components
import dash_leaflet as dl
from dash import dcc, html
import plotly.express as px
from dash import dash_table
from dash.dependencies import Input, Output, State
import base64
JupyterDash.infer_jupyter_proxy_config()

# Configure OS routines
import os

# Configure the plotting routines
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


#### FIX ME #####
# change animal_shelter and AnimalShelter to match your CRUD Python module file name and class name
from CRUD_Python_Module import AnimalShelter

###########################
# Data Manipulation / Model
###########################
# FIX ME update with your username and password and CRUD Python module name

username = "aacuser"
password = "Butters"

# Connect to database via CRUD Module
db = AnimalShelter(username, password)

# class read method must support return of list object and accept projection json input
# sending the read method an empty document requests all documents be returned
df = pd.DataFrame.from_records(db.read({}))

# MongoDB v5+ is going to return the '_id' column and that is going to have an
# invlaid object type of 'ObjectID' - which will cause the data_table to crash - so we remove
# it in the dataframe here. The df.drop command allows us to drop the column. If we do not set
# inplace=True - it will reeturn a new dataframe that does not contain the dropped column(s)
```

[16] Python

CRUD_Python_Module.py 1

C: > Users > jenni > OneDrive - SNHU > Documents > CS 340 > CRUD_Python_Module.py

```python
from pymongo import MongoClient


class AnimalShelter:
    """CRUD operations for the AAC MongoDB animals collection."""

    # initialize the MongoDB connection
    def __init__(self, username, password):
        """Connect to MongoDB using the provided credentials."""

        self.client = MongoClient(
            f"mongodb://{username}:{password}@localhost:27017/aac?authSource=aac"
        )

        self.database = self.client["aac"]
        self.collection = self.database["animals"]

    # create a new document
    def create(self, data):
        """Insert a document and return True if successful."""

        if data is not None:
            try:
                result = self.collection.insert_one(data)
                return result.acknowledged
            except Exception as e:
                print("Create error:", e)

        return False

    # read matching documents
    def read(self, query):
        """Return a list of documents matching the query."""

        if query is not None:
            try:
                results = self.collection.find(query)
                # Convert the results to a list and return it
                return list(results)
            except Exception as e:
                print("Read error:", e)

        return []

    # update matching documents
    def update(self, query, new_values):
        """Update documents and return the number modified."""

```



EXISTING FUNCTIONALITY: DASHBOARD INTERFACE

```
# define the main dashboard layout and its components
app.layout = html.Div([
    # display the grazioso salvare logo and dashboard heading
    html.Div([
        html.Img(
            src='{}'.format(encoded_image.decode()),
            style={'height': '120px'}
        ),
        html.Center(html.B(html.H1('CS-340 Grazioso Salvare Dashboard'))),
        html.Center(html.H3('Jennifer J.'))
    ]),

    html.Hr(),
    html.Div(

        # allow users to filter animals by rescue assignment type
        dcc.RadioItems(
            id='filter-type',
            options=[
                {'label': 'Water Rescue', 'value': 'water'},
                {'label': 'Mountain or Wilderness Rescue', 'value': 'mountain'},
                {'label': 'Disaster or Individual Tracking', 'value': 'disaster'},
                {'label': 'Reset', 'value': 'reset'}
            ],
            # set the default selection to display all animal records
            value='reset',
            labelStyle={'display': 'inline-block', 'margin-right': '20px'}
        )
```



EXISTING FUNCTIONALITY: INTERACTIVE DATA TABLE

```
92
93      # display animal shelter records in an interactive data table
94      dash_table.DataTable(id='datatable-id',
95                           columns=[{"name": i, "id": i, "deletable": False, "selectable": True} for i in df.columns],
96                           data=df.to_dict('records'),
97                          # enable pagination, sorting, filtering, and row selection
98                           page_size=10,
99                           sort_action="native",
100                          filter_action="native",
101                          row_selectable="single",
102                          selected_rows=[0],
103                          style_table={'overflowX': 'auto'},
104
105                          style_cell={
106                              'textAlign': 'left',
107                              'minWidth': '100px',
108                              'width': '150px',
109                              'maxWidth': '200px',
110                              'whiteSpace': 'normal'
111                          },
112                          style_header={
113                              'backgroundColor': 'rgb(230, 230, 230)',
114                              'fontWeight': 'bold'
115                          }
116
117                         ),
```



CODE ANALYSIS: APPLICATION STRUCTURE

```
2
3   # manage database operations for animal shelter records
4   class AnimalShelter:
5       """CRUD operations for the AAC MongoDB animals collection."""
6
7       # initialize the mongodb connection using supplied credentials
8       def __init__(self, username, password):
9           """Connect to MongoDB using the provided credentials."""
10
11          self.client = MongoClient(
12              f"mongodb://{username}:{password}@localhost:27017/aac?authSource=aac"
13          )
14          # select the animal shelter database and collection
15          self.database = self.client["aac"]
16          self.collection = self.database["animals"]
17
```



CODE ANALYSIS: FILTERING CALLBACK

```
142
143 # update the data table when a rescue category is selected
144 @app.callback(Output('datatable-id','data'),
145               [Input('filter-type', 'value')])
146 def update_dashboard(filter_type):
147     # filter dogs by breed, sex, and age for water rescue
148     if filter_type == 'water':
149         query = {
150             "animal_type": "Dog",
151             "breed": {
152                 "$regex": "Labrador Retriever|Chesapeake Bay Retriever|Newfoundland",
153                 "$options": "i"
154             },
155             "sex_upon_outcome": "Intact Female",
156             "age_upon_outcome_in_weeks": {"$gte": 26, "$lte": 156}
157         }
158
```



PLANNED ENHANCEMENTS: SOFTWARE DESIGN & ENGINEERING

```
18
19      # insert a new animal record into the mongodb collection
20      def create(self, data):
21          """Insert a document and return True if successful."""
22
23          # check for missing input before attempting insertion
24          if data is not None:
25              try:
26                  result = self.collection.insert_one(data)
27                  return result.acknowledged
28              # handle database errors and report unsuccessful operations
29              except Exception as e:
30                  print("Create error:", e)
31
32          return False
33
```



ALGORITHMS AND DATA STRUCTURES: EXISTING FILTERING LOGIC

```
159     # filter dogs by breed, sex, and age for mountain rescue
160     elif filter_type == 'mountain':
161         query = {
162             "animal_type": "Dog",
163             "breed": {
164                 "$regex": "German Shepherd|Alaskan Malamute|Old English Sheepdog|Siberian Husky|Rottweiler",
165                 "$options": "i"
166             },
167             "sex_upon_outcome": "Intact Male",
168             "age_upon_outcome_in_weeks": {"$gte": 26, "$lte": 156}
169         }
170 
171     # filter dogs by breed, sex, and age for disaster tracking
172     elif filter_type == 'disaster':
173         query = {
174             "animal_type": "Dog",
175             "breed": {
176                 "$regex": "Doberman Pinscher|German Shepherd|Golden Retriever|Bloodhound|Rottweiler",
177                 "$options": "i"
178             },
179             "sex_upon_outcome": "Intact Male",
180             "age_upon_outcome_in_weeks": {"$gte": 20, "$lte": 300}
181         }
182 
183     # remove rescue filters to display all animal records
184     else:
185         query = {}
```



PLANNED ENHANCEMENTS: ALGORITHMS & DATA STRUCTURES

```
206 def update_graphs(viewData):
207 
208     if viewData is None or len(viewData) == 0:
209         return [html.P("No data available for the selected filter.")]
210 
211     dff = pd.DataFrame.from_dict(viewData)
212 
213     # count the ten most common breeds in the filtered records
214     breed_counts = (
215         dff['breed']
216         .value_counts()
217         .head(10)
218         .reset_index()
219     )
220 
221     breed_counts.columns = ['breed', 'count']
222 
223     # display breed frequencies in a bar chart
224     return [
225         dcc.Graph(
226             figure=px.bar(
227                 breed_counts,
228                 x='breed',
229                 y='count',
230                 title='Top 10 Breeds'
231             )
232         )
233     ]
```



DATABASES: MONGODB READ OPERATIONS

```
34      # retrieve animal records matching the mongodb query
35      def read(self, query):
36          """Return a list of documents matching the query."""
37
38          # verify a query was provided before retrieving records
39          if query is not None:
40              try:
41                  # execute the query and return matching records as a list
42                  results = self.collection.find(query)
43                  return list(results)
44              # handle database errors during record retrieval
45              except Exception as e:
46                  print("Read error:", e)
47
48          return []
```



CODE ANALYSIS: DATABASE UPDATES AND VALIDATION

```
50      # update animal records matching the supplied query
51      def update(self, query, new_values):
52          """Update documents and return the number modified."""
53
54          # verify the query and updated values were provided
55          if query is not None and new_values is not None:
56              try:
57                  result = self.collection.update_many(query,{"$set": new_values})
58                  return result.modified_count
59              except Exception as e:
60                  print("Update error:", e)
61
62          return 0
63
64      # delete animal records matching the supplied query
65      def delete(self, query):
66          """Delete documents and return the number deleted."""
67
68          if query is not None:
69              try:
70                  # remove all records that match the query
71                  result = self.collection.delete_many(query)
72                  return result.deleted_count
73              except Exception as e:
74                  print("Delete error:", e)
75
76          return 0
```

PLANNED ENHANCEMENTS: DATABASE SECURITY & OPTIMIZATION

```
29
30  # configure credentials for the mongodb connection
31  username = "aacuser"
32  password = "Butters"
33
34  # establish the database connection through the crud module
35  db = AnimalShelter(username, password)
36
37  # retrieve animal records and convert them into a dataframe
38  df = pd.DataFrame.from_records(db.read({}))
39
40  # remove mongodb object ids before displaying records in the dashboard
41  df.drop(columns=['_id'],inplace=True)
42
```

GRAZIOSO SALVARE DASHBOARD ENHANCEMENT ROADMAP

Software Design & Engineering

Modular architecture

Record management

Automated testing

Algorithms & Data Structures

Weighted scoring

Min-heap ranking

Performance analysis

Databases

Query optimization

Schema validation

Secure access

Improved Animal Shelter Dashboard

Maintainability · Efficiency · Reliability · Security

BUTTERS

MY RESCUE















